

Algoritmos y estructuras de datos

Archivos

Archivos de texto

Para trabajar con archivos debo incluir la biblioteca `stdio.h`, donde están todas las funciones de manejo de entrada y salida de datos.

El archivo de texto no es ni más ni menos que un texto. Yo puedo generar un archivo de texto en C y después abrirlo con un bloc de notas o viceversa.

En este tipo de archivos se pueden trabajar carácter a carácter, por línea o con un formato de tipo de datos.

Si se trabaja carácter a carácter se utiliza la siguiente función para leer:

LeerCaracter `c = fgetc(f)`. F es el archivo de texto

Y para grabar un carácter:

GrabarCaracter `fputc(c,f)`. C es el carácter que guardo y F es el archivo de texto

Lo que se hace es traer a memoria lo que está en el archivo.

Si quiero traer toda una cadena se usa la siguiente función:

LeerCadena `fgets(s,n,f)`. Con s le indico la cadena donde quiero que se guarde en memoria, con n la cantidad de caracteres para esa cadena y con f el archivo.

Si quiero guardar una cadena se usa la siguiente función:

GrabarCadena `fputs(s,f)`

Finalmente, para realizar dichas funciones, pero con formato se utilizan las siguientes funciones
LeerConFormato

```
/*
LeerCaracter c = fgetc(f)
LeerCadena fgets(s,10,f)
LeerConFormato fscanf(f,"%c%f%s",&c,&r,s)
GrabarCaracter fputc(c,f)
GrabarCadena fputs(s,f)
GrabarConFormato fprintf(f,"%c %f %s \n", c,r,s)
*/
```

Cuando trabajo con un archivo, ya sea de texto o de datos, voy a definir una variable de tipo FILE* con puntero. Luego declaro la cadena y luego le doy valor a la variable de tipo archivo. Después abro el archivo con la función fopen("texto.txt", "w"), que lo que me devuelve es un puntero al archivo, es decir, va a apuntar al archivo. La primera cadena de la función corresponde al nombre del archivo, y la segunda, al tipo de apertura que le estamos dando. Existen 3 tipos de aperturas:

w: escribir. Si abrimos con w se genera un archivo nuevo

r: lectura. El archivo tiene que existir para ser leído

a: agregar. El archivo puede existir o no. Si abro el archivo con a, el puntero se va a ubicar al final del mismo.

Con formato

```
int legajo;
int nota;
f = fopen("Alumnos.txt", "w");
if (f==NULL)
    cout<<"error";
else
{
    cout<<"Ingrese legajo (0 para finalizar) ";
    cin>>legajo;
    while (legajo!=0)
    {
        cout<<"Nombre: ";
        //cin.ignore();
        //fflush(stdin);
        //gets(nom);
        //cin>>nom;
        cin.getline(nom, 31);
        cout<<"Nota: ";
        cin>>nota;
        fprintf(f, "%d |%s |d\n", legajo, nom, nota);
        cout<<"Ingrese legajo (0 para finalizar) ";
        cin>>legajo;
    }
    fclose(f);
}
```

```

void mostrar()
{
    FILE *archivo;
    char nom[31];
    int legajo;
    int nota;
    archivo = fopen("Alumnos.txt", "r");
    if (archivo == NULL)
        cout << "error";
    else
    {
        fscanf(archivo, "%d %s %d", &legajo, nom, &nota);
        while (!feof(archivo))
        {
            cout << nom << " con legajo " << legajo << " con nota " << nota << endl;
            fscanf(archivo, "%d %s %d", &legajo, nom, &nota);
        }
        fclose(archivo);
    }
}

```

Con la función `fprintf` estoy guardando con formato. La `d` es entero, `s` es una cadena y `\n` significa salto de línea. Todos se ponen con porcentaje y espacio.

Este programa no me permite ingresar el nombre porque el `enter` lo va a tomar como el ingreso del mismo, puesto que el buffer no se limpia y queda con lo que se ingresó anteriormente. Para que el buffer se limpie y el programa permite ingresar el nombre se agrega lo que está en gris. Se puede poner directamente el `cin.ignore` o sino todo el resto de líneas que están en gris.

Cuando se muestra el archivo, se utiliza la función `feof` indica la marca de fin de archivo cuando es verdadero, entonces si no es verdadero, se va a proceder a leer el archivo. Esto se hace con un `while`.

Archivos de datos

Un archivo de datos me sirve para almacenar datos, al igual que un vector, y me permite la persistencia de esos datos. C en realidad no maneja archivos de datos, sino que maneja archivos binarios, que me permiten justamente almacenar datos.

Lo primero que hacemos es definir el archivo `FILE *archivo` y le voy a dar un valor poniendo `=fopen("Nombre del archivo.extensión"`. Las extensiones que podemos ponerle `.dat` o `.bin` y luego la apertura, que son la misma que de datos pero con una `b`, es decir `wb`, `rb` o `ab`. Si el archivo deseo guardarlo en una carpeta determinada tengo que indicarle el path de la misma al principio del nombre.

Si no puede crear el archivo por algún motivo, me devuelve `NULL`. Por eso se debe preguntar si el archivo es igual a `NULL`.

Para guardar algo en el archivo debo usar la función `fwrite`, que necesita 4 parámetros. El primero es en qué dirección de memoria está lo que yo quiero guardar en el archivo. Para ello pongo el `&` que me devuelve la dirección de memoria de cualquier variable. El segundo parámetro me dice cuantos bytes quiero guardar en el archivo. Utilizo `sizeof(tipo o variable)`. El tercer parámetro me indica cuantos quiero guardar, y el cuarto, en donde quiero guardar, en este caso en la variable `archivo`.

Una vez que ingreso todo el puntero se desplaza hacia el final y luego se ingresan datos desde donde está el puntero y se vuelve a desplazar hacia el final.

Grabar archivo

```
using namespace std;

struct Parcial
{
    int legajo;
    char nombre[36];
    int nota;
};

int main()
{
    FILE *arch=fopen("Parcial.dat", "wb");
    if(arch==NULL)
        cout<<"ERROR"<<endl;
    else
    {
        Parcial parcial;
        cout<<"Legajo (cero fin) ";
        cin>>parcial.legajo;
        while(parcial.legajo!=0)
        {
            cout<<"Nombre ";
            cin.ignore();
            cin.getline(parcial.nombre,35);
            cout<<"Nota ";
            cin>>parcial.nota;
            fwrite(&parcial, sizeof(Parcial), 1, arch);
            cout<<"Legajo (cero fin) ";
            cin>>parcial.legajo;
        }
        fclose(arch);
    }
    return 0;
}
```

Leer archivo

En este caso se utiliza la función fread, que me permite traer del archivo datos a memoria, es decir, leer el archivo. Para eso necesito una variable donde voy a traer esos datos. El primer parámetro que tiene la función fread es en qué dirección de memoria quiero que lo traiga. Para ello pongo el & que me devuelve la dirección de memoria de cualquier variable. El segundo parámetro me dice cuantos bytes quiero traer del archivo a memoria. Utilizo sizeof(tipo o variable). El tercer parámetro me indica cuantos quiero traer, y el cuarto es el archivo.

El fread, al igual que el fwrite, desplaza el puntero. Pero yo no sé qué tan extenso es el archivo. Utilizo la función feof que me devuelve verdadero si al leer se detectó la marca de fin de archivo.

Si esto pasa, quiere decir que no pudo leer. Entonces hago un ciclo while que lea mientras no llegue al fin de archivo

```
{
    int leg;
    char nom[36];
    int nota;
};

int main()
{
    FILE *a=fopen("Parcial.dat", "rb");
    if(a==NULL)
        cout<<"ERROR"<<endl;
    else
    {
        Examen ex;
        fread(&ex, sizeof(Examen) 1, a);
        while(!feof(a))
        {
            cout<<ex.leg<<ex.nom<<ex.nota<<endl;
            fread(&ex, sizeof(Examen) 1, a);
        }
        fclose(a);
    }
    return 0;
}
```

Se puede indicar al principio del programa el nombre del archivo, para poder de esta manera crear diferentes archivos con la misma funcionalidad, por ejemplo, parcial 1 y parcial 2. Para eso se utiliza un vector de char y una función que concatene el .dat.

```
int main()
{
    char nombArch[21];
    cout<<"Ingrese el nombre del archivo: ";
    cin>>nombArch;
    strcat(nombArch, ".dat");
    //FILE *archParcial=fopen("Parcial1.dat", "wb");
    FILE *archParcial=fopen(nombArch, "wb");
    if(archParcial==NULL)
        cout<<"ERROR"<<endl;
}
```

Lo mismo se puede hacer para mostrar archivos.

```

int main()
{
    char nombArch[21];
    cout<<"Ingrese el nombre del archivo: ";
    cin>>nombArch;
    strcat(nombArch, ".dat");
    //FILE *a=fopen("Parcial1.dat", "rb");
    FILE *a=fopen(nombArch, "rb");
    if(a==NULL)
        cout<<"ERROR"<<endl;
    else
    {
        Examen ex;
        fread(&ex, sizeof(Examen), 1, a);
        while(!feof(a))
        {
            cout<<ex.leg<<ex.nom<<ex.nota<<endl;
            fread(&ex, sizeof(Examen), 1, a);
        }
        fclose(a);
    }
}

```

Agregar cosas a un archivo

Para agregar cosas a un archivo existente se utiliza el tipo de apertura ab, que va a mandar el puntero al final del archivo para agregarle continuar escribiendo en el archivo. Si el archivo no existe, lo genera. Solo se le cambia la forma de apertura respecto del anterior.

Mostrar un archivo ordenado

Para ordenar un archivo siempre conviene trabajarlo por fuera del mismo. Entonces, lo que se debe hacer es copiar los datos del archivo a un vector y ordenar el vector. Para ello, necesito conocer el número de registros que hay en el archivo.

Voy a definir un vector con una longitud aproximada al máximo. Luego voy a abrir el archivo, que será de solo lectura y hago una función que me copie todo lo del archivo al vector. A esta función le voy a mandar el vector y el archivo, y voy a necesitar que me devuelva la cantidad. Después lo voy a ordenar y finalmente, mostrar y cerrar el archivo.

En la función cargarVector lo que se va a hacer es leer el archivo y, mientras lo pueda leer, llenamos el v[i] haciéndolo igual a la variable que le queremos meter, y luego aumentamos i++.

```

int cargarVector(Parcial v[], FILE *a) {
    Parcial ex;
    int i=0;
    fread(&ex, sizeof(Parcial), 1, a);
    while(!feof(a))
    {
        v[i]=ex;
        i++;
        fread(&ex, sizeof(Examen), 1, a);
    }
    return i;
}

```

Si se sabe la cantidad de registros del archivo entonces se utiliza esta otra función:

```

void cargarVecConCantidad(Parcial v[], FILE *a)
{
    fread(v, sizeof(Parcial), 5, a);
}

```

Si quiero ordenar un archivo y volverlo a escribir en el archivo lo que se hace es traer ese archivo a memoria con un vector, ordenar dicho vector, y luego sobrescribir el archivo.

Primero abro el archivo, copio todo al vector, cierro el archivo y después ordeno el vector y lo sobrescribo

```

int main()
{
    Parcial vec[40]; //40 es la cantidad de alumnos, por lo tanto es la cantidad max de parciales
    FILE *arch=fopen("Parcial1.dat", "rb");
    if(arch!=NULL)
    {
        int cant=cargarVector(vec, arch);
        fclose(arch);
        ordenar(vec, cant);
        grabarArch(vec, cant);
    }

    return 0;
}

```

Para ordenar el archivo voy a usar la función grabarArch, a la cual le voy a mandar el parcial y la cantidad, y le voy a pisar el archivo que ya tenía, es decir, lo voy a sobrescribir con los datos ordenados. Para sobrescribirlo necesito abrirlo con wb. Es por esto que el archivo se cierra apenas se cargan los datos en el vector, porque no puedo abrir un archivo que ya estaba abierto. Después lo que voy a hacer es recorrer el vector y sobrescribo elemento por elemento con fwrite y un ciclo for.


```

void grabarArch(Parcial v[],int c)
{
    FILE *a=fopen("Parcial1.dat", "wb");
    for(int i=0;i<c;i++)
        fwrite(&v[i], sizeof(Parcial), 1, a);
    fclose(a);
}

```

Ahora, si los datos los ingreso desordenados, pero quiero que el archivo se genere ordenado, Para eso lo que debo hacer es cargar los datos en un vector, ordenarlos y al final de todo los debo cargar en el archivo. Necesito conocer la cantidad de datos aproximada o exacta.

Buscar en un archivo

Dado un código de producto informar su stock.

Cuando abrimos el archivo, el puntero se ubica al principio del archivo. Ingreso el código de producto a buscar y tenemos que ir leyendo uno a uno los elementos hasta que hayamos leído el que tiene el código que estamos buscando. Hago un fread en la variable &prod, sizeof(producto), 1, arch. Entonces voy a tener un ciclo while(prod.cod!=cod), mientras el producto que lei sea distinto que lo que busco, vuelva a leer.

```

int main()
{
    FILE *arch;
    arch=fopen("Productos.dat", "rb");
    if(arch==NULL)
        cout<<"ERROR";
    else
    {
        unsigned unCod;
        Producto prod;
        cout<<"Codigo de producto a buscar ";
        cin>>unCod;
        fread(&prod, sizeof(Producto), 1, arch);
        while(!feof(arch) && prod.codigo!=unCod)
            fread(&prod, sizeof(Producto), 1, arch);
        if(feof(arch))
            cout<<"no hay un producto con ese codigo"<<endl;
        else
            cout<<"el stock del producto buscado es "<<endl;
        fclose(arch);
    }
}

```

Una vez que salgo en la variable prod me queda lo que estaba buscando.

Puede pasar que lo que yo estaba buscando no esté en el archivo. En ese caso me va a quedar algo guardado, entonces debemos agregar la condición de que haga el ciclo mientras no llegue a fin de archivo. Luego debe averiguar por qué salió del ciclo, si salió porque detectó la marca de fin de archivo, aviso que no hay producto con ese código, sino, el producto fue encontrado

Modificar archivo

Podemos pedirle al programa modificar el stock de cierto producto con su código, sumándole las unidades compradas.

Lo que hacemos es abrir el archivo con "r+b" o "rb+". Le agrega la funcionalidad que no tiene, si le pongo el + es de lectura y escritura. Voy a reescribir un registro que ya existía, modificar uno ya existente. No voy a agregarle nada al final del registro, como se hace con el ab. Es una modificación, no un agregado. Vamos a buscar y sobrescribir cierto registro. Utilizo el mismo ciclo de búsqueda con el while que en el caso anterior.

Si el producto se encuentra, lo que debemos hacer es modificarlo haciendo `prod.stock+=uCompra`. Con esto suma en memoria, no en el archivo. Lo que debo hacer entonces es reescribir el producto entero del registro. Tenemos que hacer un `fwrite(&prod, sizeof(Producto), 1, arch)` para sobrescribir. Ojo porque el puntero me quedo al final del registro que quería modificar. Para evitar esto debemos traer el puntero un producto atrás, entonces usamos una función `fseek`. Tiene tres parámetros, el primero es el archivo, el tercer es desde donde yo quiero que se mueva el puntero: desde 0 o desde `SEEK_SET` (es lo mismo que 0), o sea, desde el principio; desde 1 o `SEEK_CUR`, que es que se mueva desde donde está ahora; desde 2 o `SEEK_END`, que es que se mueva desde el final. El segundo parámetro es cuántos bytes yo quiero que desplace. Esa cantidad de bytes puede ser positiva (hacia el final del archivo) o negativa (hacia el comienzo del archivo). La función devuelve cuánto se movió, si no se pudo mover devuelve 0.

Para correr el producto hacia atrás hacemos `fseek(arch, -sizeof(Producto), SEEK_CUR)`. Luego si que vamos a reescribir.

```
FILE *arch;
arch=fopen("Productos.dat", "r+b");
if (arch==NULL)
    cout<<"ERROR";
else
{
    unsigned unCod, uCompra;
    Producto prod;
    cout<<"Codigo a buscar ";
    cin>>unCod;
    cout<<"Unidades compradas para sumar al stock ";
    cin>>uCompra;
    fread(&prod, sizeof(Producto), 1, arch);
    while (!feof(arch) && prod.codigo!=unCod)
        fread(&prod, sizeof(Producto), 1, arch);
    if (feof(arch))
        cout<<"no hay un producto con ese codigo"<<endl;
    else
    {
        prod.stock+=uCompra;
        //correr el puntero un Producto para atras
        fseek(arch, -sizeof(Producto), SEEK_CUR);
        fwrite(&prod, sizeof(Producto), 1, arch);
    }
}
```

Le voy a agregar una funcionalidad mediante la cual, si no encontró el producto, se lo agrego al final del archivo con un `fwrite` (sin `ab` porque el puntero ya está en el final del archivo) que se pone después de definir un código y un stock.

```

        cout<<"no hay un producto con ese codigo";
        cout<<"quiere agregarlo? (S/N)";
        char resp;
        cin>>resp;
        if(resp=='S')
        {
            prod.codigo=unCod;
            prod.stock=uCompra;
            fwrite(&prod,sizeof(Producto),1,arch);
        }
    }
    else
    {
        prod.stock+=uCompra;
        //correr el puntero un Producto para atr
        fseek(arch,-sizeof(Producto),SEEK_CUR);
        fwrite(&prod,sizeof(Producto),1,arch);
    }

    fclose(arch);
}
return 0;
}

```

Lo ideal es trabajar todas estas cuestiones con funciones.

Funciones de búsqueda en archivos

```

int busquedaSecuencial(FILE* arch,unsigned bus,Producto &prod)
{
    unsigned i=0;
    fread(&prod,sizeof(Producto),1,arch);
    while(!feof(arch)&&prod.codigo!=bus)
    {
        fread(&prod,sizeof(Producto),1,arch);
        i++;
    }
    if(feof(arch))
        return -1;
    else
        return i;
}

```

Recibe el archivo, lo que tiene que buscar y hago que devuelva la posición del registro y el producto en un parámetro por referencia.

El main queda modificado tal que así:

```

        cout<<"Codigo de producto a buscar ";
        cin>>unCod;
        int pos=busquedaSecuencial(arch,unCod);
        if(pos==-1)
            cout<<"no hay un producto con ese codigo"<<endl;
        else
        {
            Producto prod;
            fseek(arch,pos*sizeof(Producto),SEEK_SET);
            fread(&prod,sizeof(Producto),1,arch);
            cout<<"el stock del producto buscado es "<<prod.stock<<endl;
        }

        fclose(arch);
    }

    return 0;
}

```

Si no me dio -1, me dio la posición del producto dentro del archivo. Para obtener el producto sin tener la variable por referencia en la función voy a necesitar una variable Producto prod y leer el archivo, yendo con fseek a la posición pos fseek(arch, pos*sizeof(Producto), SEEK_SET) y haciendo un fread.

Es mejor hacer que la función me devuelva por parámetro el producto, entonces de esta manera no es necesario hacer el fseek y el fread

```

//CONSULTA
FILE *arch;
arch=fopen("Productos.dat","rb");
if(arch==NULL)
    cout<<"ERROR";
else
{
    unsigned unCod;
    Producto prod;
    cout<<"Codigo de producto a buscar ";
    cin>>unCod;
    int pos=busquedaSecuencial(arch,unCod,prod);
    if(pos==-1)
        cout<<"no hay un producto con ese codigo"<<endl;
    else
    {
        /*
        fseek(arch,pos*sizeof(Producto),SEEK_SET);
        fread(&prod,sizeof(Producto),1,arch);
        */
        cout<<"el stock del producto buscado es "<<prod.stock<<endl;
    }

    fclose(arch);
}

```

```

cin>>uCompra;
int pos=busquedaSecuencial (arch,unCod,prod);
if(pos!=-1)
{
    cout<<"no hay un producto con ese codigo"<<endl;
    cout<<"quiere agregarlo? (S/N)";
    char resp;
    cin>>resp;
    if(resp=='S')
    {
        prod.codigo=unCod;
        prod.stock+=uCompra;
        fseek(arch,0,SEEK_END);
        fwrite(&prod,sizeof(Producto),1,arch);
    }
}
else
{
    prod.stock+=uCompra;
    //correr el puntero un Producto para atras
    fseek(arch,pos*sizeof(Producto),SEEK_SET);
    fwrite(&prod,sizeof(Producto),1,arch);
}

fclose(arch);
}

```

La función fseek dentro del if es para ubicar el puntero al principio del archivo. (VER CLASE Y CORREGIR)

Búsqueda binaria en archivos

Si tengo el archivo ordenado se puede hacer una búsqueda binaria.

```

int busquedaBinaria(FILE* arch,unsigned bus,Producto &prod)
{
    unsigned desde,hasta,medio,cantRegistros;
    int pos=-1;
    desde=0;
    fseek(arch,0,SEEK_END);
    cantRegistros=ftell(arch)/sizeof(Producto);
    hasta=cantRegistros-1;
    //hasta=ftell(arch)/sizeof(Producto)-1;
    while(pos!=-1 && desde<=hasta)
    {
        medio=(desde+hasta)/2;
        fseek(arch,medio*sizeof(Producto),SEEK_SET);
        fread(&prod,sizeof(Producto),1,arch);
        if(prod.codigo==bus)
            pos=medio;
        else
        {
            if(bus<prod.codigo)
                hasta=medio-1;
            else
                desde=medio+1;
        }
    }
    return pos;
}

```

Para saber la cantidad de registros se utiliza la función ftell(arch)/sizeof(Producto) que lo que me devuelve es la cantidad de bytes que hay desde donde empieza el archivo hasta donde está el puntero en ese momento. Para eso ubico antes el puntero al final del archivo con la función fseek. Al dividirlo por la cantidad de bytes del producto entonces me devuelve el registro.

Acceso directo en archivos

Estuvimos trabajando los archivos con la misma lógica que los vectores. Entonces también podemos trabajarlo como si fuese acceso directo, estableciendo relación entre las posiciones y código de producto. Entonces cuando hago la búsqueda accedo directamente a la posición sin hacer ninguna búsqueda binaria o secuencial.

Evalúo primero si el código a buscar está entre los valores que están definidos en el vector. Entonces como ya se donde está hago un `fseek(aProd, (unCod-1)*sizeof(Producto), SEEK_SET)`. Con esto me muevo hasta el producto que me interesa y después lo leo con un `fread(&p, sizeof(Producto), 1, aProd)` y luego un `cout` mostrando `p.stock`.

```
int main()
{
    FILE *aProd;
    Producto p;
    aProd=fopen("ProductosAccDir.dat", "rb");
    cout<<"Codigo de producto a buscar ";
    int unCod;
    cin>>unCod;
    if (aProd!=NULL)
    {
        if(unCod>=1 && unCod<=10)
        {
            fseek(aProd, (unCod-1)*sizeof(Producto), SEEK_
            fread(&p, sizeof(Producto), 1, aProd);
            cout<<p.stock<<endl;
        }
        else
    }
```

Apareo en archivos

No necesariamente se debe hacer con dos archivos, es decir, se puede hacer un apareo de un archivo con un vector, siempre y cuando se cumplan las reglas necesarias para que se cumpla el apareo.

Ahora vamos a hacer un apareo de dos archivos. En el abrimos los dos archivos:

```
string condicion(unsigned n1, unsigned n2);
void apareo(FILE*ap1, FILE*ap2);
void corteControl(FILE*a);

int main()
{
    FILE *p1=fopen("Parcial1.dat", "rb");
    if(p1!=NULL)
    {
        FILE *p2=fopen("Parcial2.dat", "rb");
        if(p2!=NULL)
        {
            cout<<endl<<"Apareo"<<endl;
            apareo(p1,p2);
            fclose(p1);
            fclose(p2);
        }
        fclose(p1);
    }
}
```

Debemos recorrer los dos archivos a la vez para ponerlos en una lista con todos los alumnos y la condición de si van a final, si promocionaron o si recursan. Los archivos los voy a recorrer mientras no lleguen a fin de archivo.

Vamos a usar la función condición para establecer la condición del alumno según las notas que obtuvo en el parcial.

```
string condicion(unsigned n1, unsigned n2)
{
    if(n1>7&& n2>7)
        return " Prom";
    else
    {
        if(n1>5&& n2>5)
            return " Final";
        else
            return " Recursa";
    }
}
```

Primero debemos leer los dos archivos. Declaramos de tipo parcial las variables p1 y p2. Luego hacemos fread del parcial 1 y fread del parcial 2. Hacemos un ciclo que se cumpla mientras que no llegue a fin de archivo en ninguno de los dos (ifeof). Ahora debemos comparar si son iguales.

Voy a preguntar si el DNI del primero es menor que el DNI del segundo, es decir, comparo los menores de ambos archivos. Si es el caso, debo mostrarlo con un cout e invoco a la función condición que me diga como le fue en ese parcial. Esta función recibe las notas de los dos parciales y devuelve la condición, pero como solo estamos tratando el primero en esta instancia, en el campo del segundo parcial le envío un 0. Ahora vuelvo a leer el archivo del parcial.

Si el DNI del primero no es menor que el del segundo, pregunto si es igual. Si ese es el caso, debo procesar los dos, mostrando cualquiera de los dos archivos, ya que son iguales. Además invoco a la función condición, a la que esta vez le enviaré la nota de ambos parciales. Luego, vuelvo a leer los dos, pues se procesaron los dos archivos.

Si el DNI del primero es mayor que el del segundo, entonces proceso el menor, es decir, proceso el segundo archivo e invoco la función condición, a la que le voy a mandar solo la nota del segundo parcial, y en el primero pongo 0. Luego vuelvo a leer el parcial 2.

Cuando se detecta la marca de fin de archivo de uno de los dos archivos, se debe procesar lo que quedó pendiente en el otro archivo. Para ello ponemos afuera del ciclo while otro ciclo while que tenga la condición (!feof) en cada uno de los archivos, y mostrar lo que está en el archivo y volver a leerlo. Se ponen los dos archivos porque no puedo estar seguro por cual de los dos archivos salí del primer ciclo while.

```

void apareo(FILE*ap1,FILE*ap2)
{
    Parcial p1,p2;
    fread(&p1,sizeof(Parcial),1,ap1);
    fread(&p2,sizeof(Parcial),1,ap2);
    while(!feof(ap1)&&!feof(ap2))
    {
        if(p1.dni<p2.dni)
        {
            cout<<p1.dni<<p1.nom<<condicion(p1.nota,0)<<endl;
            fread(&p1,sizeof(Parcial),1,ap1);
        }
        else
        {
            if(p1.dni==p2.dni)
            {
                cout<<p1.dni<<p1.nom<<condicion(p1.nota,p2.nota)<<endl;
                fread(&p1,sizeof(Parcial),1,ap1);
                fread(&p2,sizeof(Parcial),1,ap2);
            }
            else
            {
                cout<<p2.dni<<p2.nom<<condicion(0,p2.nota)<<endl;
                fread(&p2,sizeof(Parcial),1,ap2);
            }
        }
    }
}

```

```

while(!feof(ap1))
{
    cout<<p1.dni<<p1.nom<<condicion(p1.nota,0)<<endl;
    fread(&p1,sizeof(Parcial),1,ap1);
}
while(!feof(ap2))
{
    cout<<p2.dni<<p2.nom<<condicion(0,p2.nota)<<endl;
    fread(&p2,sizeof(Parcial),1,ap2);
}
}

```

Corte de control en archivos

Recordemos que el corte de control se hace cuando tengo datos repetidos y agrupados. En este caso haremos un listado con los alumnos y su promedio. Para cada alumno tengo que procesar sus exámenes y cuando termino un alumno, lo muestro. Entonces, el corte de control consta de dos ciclos: el primero, donde voy a ir al primero de cada grupo; y el segundo, donde voy a ir recorriendo todos los exámenes de cada alumno.

En el main pongo lo siguiente:


```

FILE *ex=fopen("Exámenes.dat","rb");
if (ex!=NULL)
{
    cout<<endl<<"Corte de control"<<endl;
    corteControl(ex);
    fclose(ex);
}

return 0;

```

Voy a crear una variable donde guardo el DNI del alumno que estoy procesando en el momento y lo voy a comparar. Además declaramos una variable de tipo parcial para poder leer el archivo. Asimismo, como vamos a calcular un promedio necesitamos un sumador y un contador, que lo declaramos en este momento.

Leemos y luego abrimos un ciclo while, donde recorreremos el archivo mientras no sea fin de archivo. Dentro del mismo guardamos en la variable dniA el DNI del alumno que estoy leyendo. Al sumador lo inicializamos en 0 dentro ya que se reinicia por cada alumno, mismo con el contador.

Ahora hacemos un ciclo do while, porque siempre va a entrar por lo menos una vez, ya que al menos leyó un examen de un alumno. Ahora hacemos suma+=nota y sumamos uno al contador. Luego voy a leer el archivo. La condición del while va a ser que el DNI que estoy procesando sea igual al DNI que guardé en la variable dniA. Además, debemos pedirle que no sea fin de archivo.

Una vez que sale del último ciclo while, quiere decir que cambió de DNI y que va a entrar a otro alumno. Entonces hago un cout para mostrar al alumno que se procesó.

```

void corteControl(FILE*a)
{
    unsigned dniA,suma,cont;
    Parcial p;
    fread(&p,sizeof(Parcial),1,a);
    while(!feof(a))
    {
        dniA=p.dni;
        suma=0;
        cont=0;
        do
        {
            suma+=p.nota;
            cont++;
            fread(&p,sizeof(Parcial),1,a);
        }while(!feof(a)&& p.dni==dniA);
        cout<<dniA<<suma/cont<<endl;
    }
}

```

Si hubiese querido mostrar el nombre, debería haberlo guardado también en otra variable a declarar al principio.